

DELIVRABLE 2

BIGDATA CHU PROJECT

G1 : Hedi / Benjamin / Jordan / Abdillahi

A3 CS 25/26

CESI 
ÉCOLE D'INGÉNIEURS



TABLE OF CONTENTS

1. Executive Summary & Architecture Overview	3
1.1. Scope of Deliverable 2 & Checkpoints Alignment.....	3
1.2. The Logical Architecture	3
1.3. ELT (Extract-Load-Transform) Strategy Justification on Cloudera.....	4
2. Physical Implementation & Data Ingestion of the Staging Layer.....	4
2.1. Source Ingestion.....	4
2.1.1. Dim_Diagnostic	5
2.1.2. Dim_etablissement	6
2.1.3. Dim_etablissement	7
2.1.4. Dim_Patient.....	9
2.1.5. Dim_professionnel.....	10
2.1.6. Dim_temps.....	12
2.1.7. Dim_Consultation	13
2.2. Architectural Justification of the Ingestion Strategy	14
3. Measurement Table Design & Calculation Logic (Fact Table)	15
3.1. Table Materialization for Measurements	15
4. Data Materialization and Operational Query Compilation in Apache Hive (Hue).	16
4.1. HiveQL Scripts.....	16
4.1.1. CreateHospitalisation & InsertHospitalisation.....	16
4.1.2. CreateConsultation & InsertConsultation.....	18
4.1.3. CreateDeaths & InsertDeaths	20
4.1.3. CreateSatisfaction & InsertSatisfaction.....	21
5. Performance Assessment & Analytical Query Analysis	26
5.1. Target Analytical Queries Execution.....	26
5.1.1. Advanced HiveQL Optimization Techniques.....	26
5.2. Query Performance Testing Methodology & Benchmarking Environment	26
5.3. Comparative Runtime Performance & Response Time Analysis.....	27
5.3.1. Technical Performance Drivers	27
5.3.2. Live MapReduce Job Log Outputs.....	28
6. Conclusion & Deliverables Index.....	29
6.1 Conclusion & technical Challenges Overcome.....	29
6.2 Next Steps: Visualization and Data Storytelling (Deliverable 3)	29
6.3. Index of Delivered Scripts (Attached .zip Manifest File).....	29
7. BIBLIOGRAPHY & REFERENCES	30
8. APPENDICES	31
8.1. Appendix A : Architecture.....	31
8.2. Appendix B : Galaxy Schema.....	32

1. Executive Summary & Architecture Overview

1.1. Scope of Deliverable 2 & Checkpoints Alignment.

This technical report covers the physical implementation, optimization, and benchmarking of Cloud Healthcare Unit (CHU) Big Data decision-making system. Having received the conceptual design in Deliverable 1, this stage is dedicated to transforming raw data into optimized Galaxy Schema with Apache Hive.

The goal is to create analytical infrastructure that would allow monitoring medical flows such as consultation or hospitalization and patient satisfaction. The system will be designed to offer quick response rates for queries from facility administrators and clinicians while processing large historical datasets.

This deliverable is tightly linked to all required evaluation milestones:

Physical Modeling Layer: The structure of the storage layer comprises six analytical dimension tables, along with four measurement tables. All the entities have been explicitly mentioned to be Hive Tables.

Data Lake & Quality: Talend Big Data jobs have been used for source schema replication and staging in HDFS before applying any analysis.

Analytical Metrics: The fact level computes 4 metrics (consultation, hospitalisation, satisfaction, deces) based on ensuring referential integrity for foreign keys.

Hardware Utilization: Tables take advantage of advanced Hive optimizations using optimized hardware partitions.

1.2. The Logical Architecture

In order to provide a good process , a coupled architecture was adopted for implementation within the Cloudera Hadoop ecosystem. The architecture achieves isolation of risks involved in operations from analytical processing via a separate database:

Data Storage & Transformation Layer (Projet): A single database named Projet is established within Apache Hive to manage our analytical assets. It acts as the operational engine where raw tables are ingested and modeled into galaxy schema patterns. This unified platform serves as the clean, pre-computed data foundation hosting the final reporting data model.

1.3. ELT (Extract-Load-Transform) Strategy Justification on Cloudera

Unlike traditional BI architectures utilizing ETL pipelines involving data manipulation within the mid-level middleware application servers, this particular design strictly employs the principles of the ELT methodology.

As per this model, data is extracted from the corresponding data stores using Talend and loaded directly into the HDFS environment without undergoing any intermediate transformation, except for some localized tMap configurations applied during the ingestion stage to handle basic mapping and structural formatting anomalies. Transformations such as data cleaning, string formatting, datatype rigidity enforcement, relation establishment, and numerical aggregation are carried out directly within Apache Hive.

The benefit of employing such ELT-based approach is in the utilization of MPP (Massive Parallel Processing) capabilities offered by Cloudera. This allows having a good performance of transformations at the storage level, avoiding bottleneck issues, scale to accommodate data growth, and maximum freedom in modifying data downstream.

2. Physical Implementation & Data Ingestion of the Staging Layer

2.1. Source Ingestion.

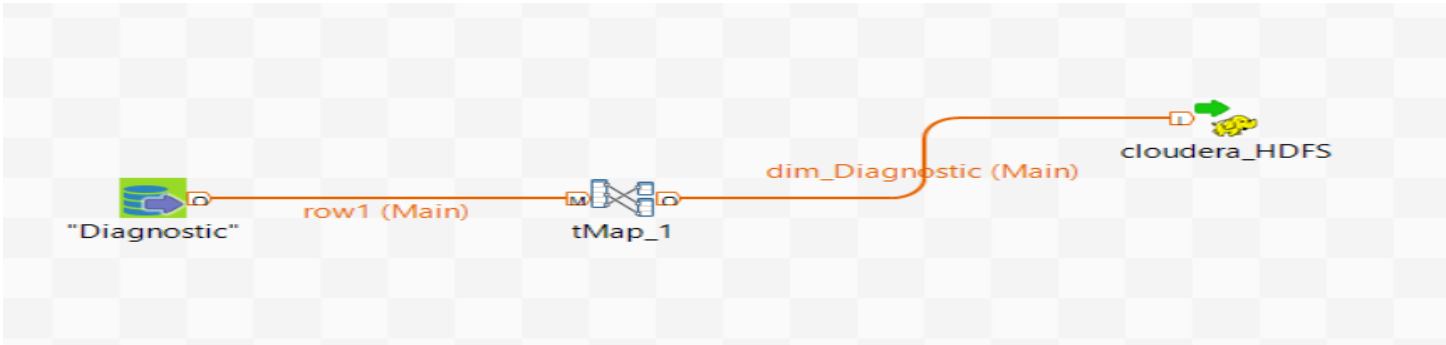
The ingestion layer collects data from transactional systems and file repositories to be ingested in the HDFS data lake. In order to ensure preservation of the true historical record and avoid data pipelines dropping out due to premature structural modifications and cleansing, the data is streamed into the isolated HDFS directory endpoints. The ingestion processes are split into two main storage sources that are handled via optimized Talend jobs:

PostgreSQL Core Relational Tables: Operational transactional data such as patient tracking, admissions, diagnostics, and consultations is extracted from a native PostgreSQL instance. Talend jobs pull from the database source and synchronize the operational data table into the HDFS layer with cloudera_HDFS target jobs.

Flat File Registries: Heterogeneous external flat files, national registries, and Excel tables are processed via optimized format adjustments in order to handle their absence of relational integrity rules:

- **etablissements:** Uses a delimiter to process the etablissement and localisation flat files and routes each row to tMap to ensure standardized format.
- **deces:** Sends delimited text files to tFilterRow transformation to remove any invalid rows before storing in HDFS.
- **Health Infrastructure & Personnel (hospitalisation, professionnel, activite):** Consume the structure properties of buildings as well as those of health professionals. These tasks perform multiple source lookups for the professionals by utilizing tMap tasks.
- **Patient Satisfaction Forms (satisfaction_hopitaux, satisfaction_v2, Enquete_satisfaction):** Consume structures from diverse sources. These jobs pass vector rows through particular mapping operations to appropriately structure the data structures into string and numeric structures to avoid down-stream data truncation problems [4].

2.1.1. Dim_Diagnostic



row1

Colonne
Code_diag
Diagnostic

Var

Expression	Type	Variable
------------	------	----------

dim_Diagnostic

Expression	Colonne
Numeric.sequence("s1", 1, 1)	id_Diag
row1.Code_diag	Code_diag
row1.Diagnostic	Diagnostic

row1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
Code_diag		String	☒		21474836...	0		
Diagnostic		String	☐		21474836...	0		

dim_Diagnostic

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
id_Diag		Int	☒					
Code_diag		String	☐		21474836...	0		
Diagnostic		String	☒		21474836...	0		

cloudera_HDFS(tHDFSOutput 1)

Job dim_diagnostic

Paramètres simples

Paramètres avancés

Paramètres dynamiques

View

Documentation

Type de propriété

Référentiel

HDFS:cloudera_HDFS

Schéma

Built-In

Modifier le schéma

Sync colonnes

Utiliser une connexion existante

Version

Distribution

Cloudera

Version

Cloudera CDH5.12(YARN mode)

Scheme

Scheme

HDFS

Connexion

URI du NameNode

hdfs://quickstart.cloudera:8020

Utiliser le nom d'hôte du nœud de données

Authentification

Utiliser l'authentification de Kerberos

Utilisateur

cloudera

Nom de fichier

/user/hive/warehouse/dim_diagnostic/dim_diagnostic.csv

Type de fichier

Type

Fichier texte

Action

Ecraser

Séparateur de lignes

\n

Séparateur de champs

,

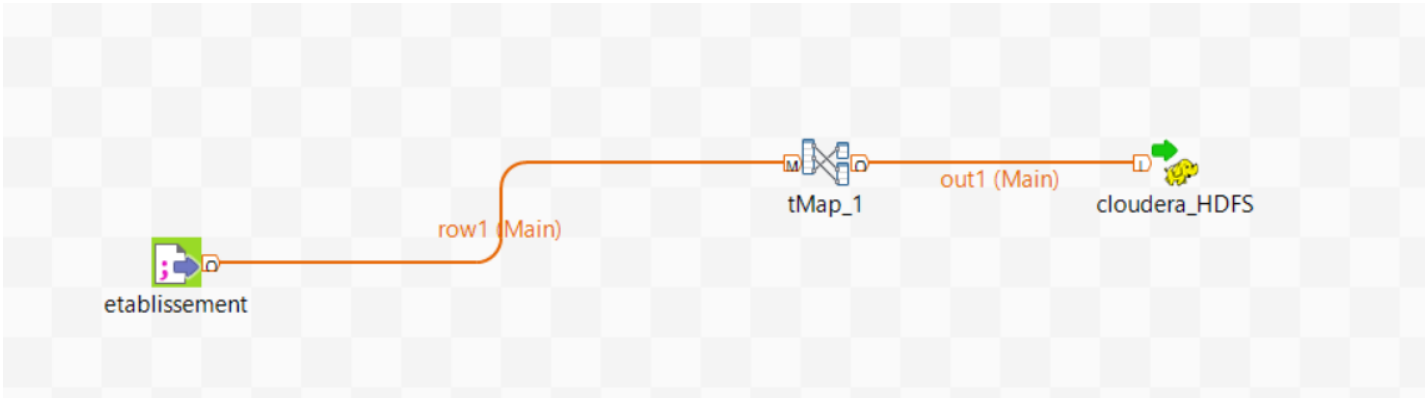
Encodage personnalisé

Compression

Compresser les données

Inclure l'en-tête

2.1.2. Dim_etablissement



row1

Column
adresse
cedex
code_commune
code_postal
commune
complement_destinataire
complement_point_geographique
email
enseigne_commerciale_site
fitness_etablissement_juridique
fitness_site
identifiant_organisation
indice_repetition_voie
mention_distribution
numero_voie
pays
raison_sociale_site
siren_site
siret_site
telecopie
telephone
telephone_2
type_voie

Var

Expression	Type	Variable
Numeric.sequence("s1",1,1).toString()	String	☐ id_etablissement

out1

Expression	Column
Var.id_etablissement	id_etablissement
row1.adresse	adresse
row1.identifiant_organisation	identifiant_organisation
row1.pays	pays
row1.siren_site	siren_site
row1.code_postal	code_postal

ow1

Colonne	Clé	Type	☒ N.	Modèle de date (Ct..	Length	Precision	Défaut	Comment...
adresse	☐	String	☒		58	0		
cedex	☐	String	☒		30	0		
code_commune	☐	String	☒		5	0		
code_postal	☐	String	☒		5	0		
commune	☐	String	☒		24	0		
complement_destinataire	☐	String	☒		38	0		

out1

Colonne	Clé	Type	☒ N.	Modèle de date (Ct..	Length	Precision	Défaut	Comment...
id_etablissement	☒	String	☐					
adresse	☐	String	☒		58	0		
identifiant_organisation	☐	String	☒		15	0		
pays	☐	String	☒		6	0		
siren_site	☐	String	☒		9	0		
code_postal	☐	String	☒		5	0		

cloudera_HDFS(thDFSOutput_1)

Paramètres simples

Paramètres avancés

Paramètres dynamiques

View

Documentation

Type de propriété **Référentiel** HDFS:cloudera_HDFS

Schéma **Built-In** Modifier le schéma Sync colonnes

☐ Utiliser une connexion existante

Version

Distribution **Cloudera** Version **Cloudera CDH5.12(YARN mode)**

Scheme

Scheme **HDFS**

Connexion

URI du NameNode **"hdfs://quickstart.cloudera:8020"**

☒ Utiliser le nom d'hôte du nœud de données

Authentification

☐ Utiliser l'authentification de Kerberos

Utilisateur **"cloudera"**

Nom de fichier **"/user/hive/warehouse/dim_etablissement/dim_etablissement.csv"**

Type de fichier

Type **Fichier texte**

Action **Ecraser**

Séparateur de lignes **"\n"** Séparateur de champs **","**

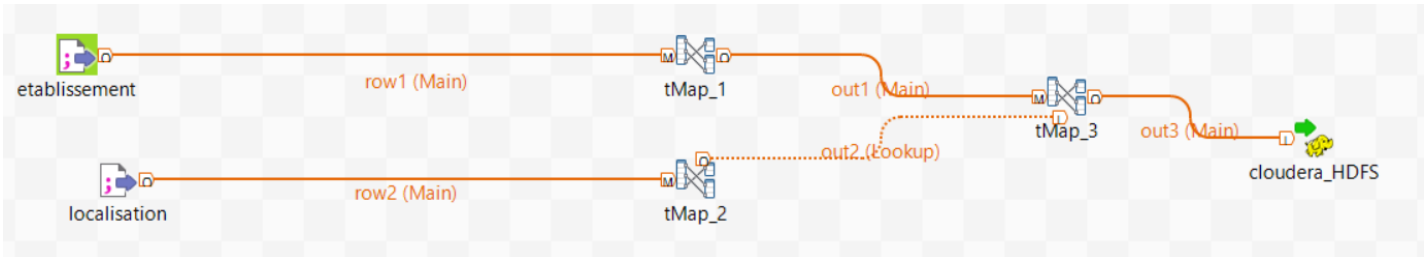
☐ Encodage personnalisé

Compression

☐ Compresser les données

☒ Inclure l'en-tête

2.1.3. Dim_localization



row1

Column

adresse

cedex

code_commune

code_postal

commune

complement_destinataire

complement_point_geographique

email

enseigne_commerciale_site

finess_etablissement_juridique

finess_site

identifiant_organisation

indice_repetition_voie

mention_distribution

numero_voie

pays

raison_sociale_site

siren_site

siret_site

telecopie

telephone

telephone_2

type_voie

Var

out1

Expression

row1.code_postal

row1.commune

row1.adresse

row1.pays

row1.finess_etablissement_juridique

row1.siren_site

row1.code_postal != null ... && { ... (row1.code_...

code_postal

commune

adresse

pays

finess_etablissement_juridique

siren_site

dep_code

Éditeur de Schéma Éditeur d'expression

row1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
adresse		String	☒		58	0		
cedex		String	☒		30	0		
code_commune		String	☒		5	0		
code_postal		String	☒		5	0		
commune		String	☒		24	0		
complement_destinataire		String	☒		38	0		

out1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
code_postal		String	☒		5	0		
commune		String	☒		24	0		
adresse		String	☒		58	0		
pays		String	☒		6	0		
finess_etablissement_juridiqu...		String	☒		9	0		
siren_site		String	☒		9	0		

row2

Column

Column0

code_insee

nom_standard

nom_sans_prenom

nom_a

nom_de

nom_sans_accent

nom_standard_majuscule

typecom

typecom_texte

reg_code

dep_code

dep_nom

canton_code

canton_nom

epci_code

epci_nom

academie_code

academie_nom

code_postal

codes_postaux

zone_emploi

Var

out2

Expression

row2.nom_standard

row2.nom_sans_accent

row2.dep_code

row2.reg_nom

row2.reg_code == null ? null : row2.reg_code.toStr...

row2.codes_postaux

row2.code_postal

nom_standard

nom_sans_accent

dep_code

reg_nom

reg_code

codes_postaux

code_postal

Éditeur de Schéma Éditeur d'expression

row2

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
Column0		String	☒		3	0		
code_insee		String	☒		5	0		
nom_standard		String	☒		24	0		
nom_sans_prenom		String	☒		22	0		
nom_a		String	☒		25	0		
nom_de		String	☒		27	0		

out2

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
nom_standard		String	☒		24	0		
nom_sans_accent		String	☒		23	0		
dep_code		String	☒		2	0		
reg_nom		String	☒		21	0		
reg_code		String	☒		2	0		
codes_postaux		String	☒		7	0		

out1

- code_postal
- commune
- adresse
- pays
- finess_etablissement_juridique
- siren_site
- dep_code

out2

Property	Value
Lookup Model	Charge une fois
Match Model	Correspondance unique
Join Model	Left Outer Join
Store temp data	false

Clé d'expr.

Column
nom_standard
nom_sans_accent
dep_code
reg_nom
reg_code
codes_postaux
code_postal

out1.dep_code

Var

Expression	Type	Variable
Numeric.sequence("s1",1,1).toString()	String	☐ id_localisation

out3

Expression	Column
out1.code_postal	code_postal
out1.commune	commune
out1.adresse	adresse
out1.pays	pays
out2.dep_code	dep_code
out2.reg_nom	reg_nom
out2.reg_code	reg_code
Var.id_localisation	id_localisation

Éditeur de Schéma
Éditeur d'expression

out1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
code_postal	☐	String	☒		5	0		
commune	☐	String	☒		24	0		
adresse	☐	String	☒		58	0		
pays	☐	String	☒		6	0		
finess_etablissement_juridiqu...	☐	String	☒		9	0		
siren_site	☐	String	☒		9	0		

out3

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
code_postal	☐	String	☒		5	0		
commune	☐	String	☒		24	0		
adresse	☐	String	☒		58	0		
pays	☐	String	☒		6	0		
dep_code	☐	String	☒		2	0		
reg_nom	☐	String	☒		21	0		

cloudera_HDFS(tHDFSOutput_1)

Paramètres simples

Paramètres avancés

Paramètres dynamiques

View

Documentation

Type de propriété

Référentiel

HDFS:cloudera_HDFS

Schéma

Built-In

Modifier le schéma

Sync colonnes

Utiliser une connexion existante

Version

Distribution

Cloudera

Version

Cloudera CDH5.12(YARN mode)

Scheme

HDFS

Connexion

URI du NameNode

hdfs://quickstart.cloudera:8020

Utiliser le nom d'hôte du nœud de données

Authentification

Utiliser l'authentification de Kerberos

Utilisateur

cloudera

Nom de fichier

/user/hive/warehouse/dim_localisation/dim_localisation.csv

Type de fichier

Type

Fichier texte

Action

Ecraser

Séparateur de lignes

\n

Séparateur de champs

,

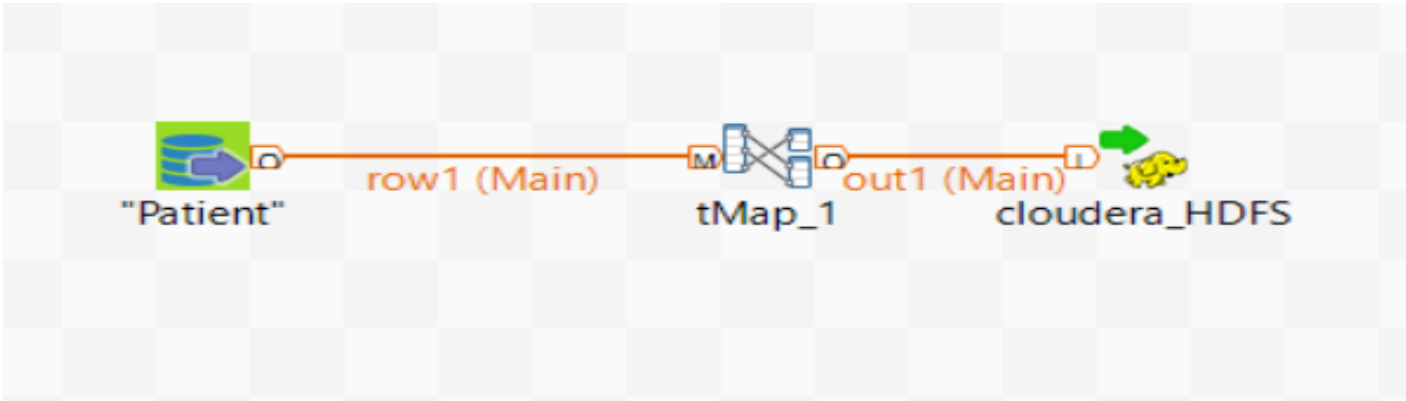
Encodage personnalisé

Compression

Compresser les données

Inclure l'en-tête

2.1.4. Dim_Patient



row1

Column
Id_patient
Nom
Prenom
Sexe
Adresse
Ville
Code_postal
Pays
EMail
Tel
Date
Age
Num_Secu
Groupe_sanguin
Poid
Taille

Var

out1

Expression	Column
row1.Id_patient	Id_patient
row1.Ville	Ville
Integer.parseInt(row1.Code_postal)	Code_postal
TalendDate.parseDate("dd/MM/yyyy", row1.Date)	Date
row1.Groupe_sanguin	Groupe_sanguin
row1.Poid	Poid
row1.Taille	Taille
row1.Age	Age
row1.Sexe	Genre

row1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
Id_patient	☒	int	☐		10	0		
Nom	☐	String	☒		20	0		
Prenom	☐	String	☒		23	0		
Sexe	☐	String	☒		6	0		
Adresse	☐	String	☒		100	0		
Ville	☐	String	☒		100	0		

out1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
Id_patient	☒	int	☐		10	0		
Ville	☐	String	☒		100	0		
Code_postal	☐	Integer	☒		21474836...	0		
Date	☐	Date	☒	"yyyy-MM-dd"	10	0		
Groupe_sanguin	☐	String	☒		3	0		
Poid	☐	String	☒		5	0		

cloudera_HDFS(tHDFSOutput_1)

Paramètres simples

Paramètres avancés

Paramètres dynamiques

View

Documentation

Type de propriété Référentiel HDFS:cloudera_HDFS

Schéma Built-In Modifier le schéma Sync colonnes

☐ Utiliser une connexion existante

Version Cloudera Version Cloudera CDH5.12(YARN mode)

Schéma HDFS

Connexion

URI du NameNode "hdfs://quickstart.cloudera:8020"

☒ Utiliser le nom d'hôte du nœud de données

Authentification

☐ Utiliser l'authentification de Kerberos

Utilisateur "cloudera"

Nom de fichier "/user/hive/warehouse/dim_patient/dim_patient.csv"

Type de fichier Fichier texte

Action Ecraser

Séparateur de lignes "\n" Séparateur de champs ","

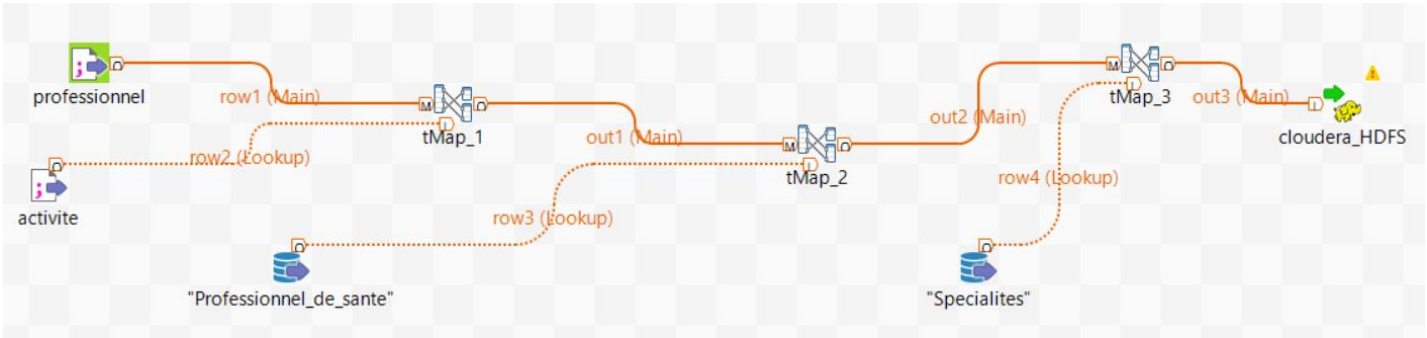
☐ Encodage personnalisé

Compression

☐ Compresser les données

☒ Inclure l'en-tête

2.1.5. Dim_professionnel



row1

Column

identifiant

civilite

categorie_professionnelle

nom

prenom

commune

profession

specialite

type_identifiant

row2

Property

Lookup Model

Match Model

Join Model

Store temp data

Clé d'expr.

row1.identifiant

Value

Charge une fois

Correspondance unique

Left Outer Join

false

Column

identifiant

identifiant_organisation

mode_exercice

Var

out1

Expression

row1.identifiant

row1.civilite

row1.categorie_professionnelle

row1.nom

row1.prenom

row1.commune

row1.profession

row1.specialite

row1.type_identifiant

row2.mode_exercice

row2.identifiant_organisation

row2.identifiant

Column

identifiant

civilite

categorie_professionnelle

nom

prenom

commune

profession

specialite

type_identifiant

mode_exercice

identifiant_organisation

identifiant_1

Editeur de Schéma

Editeur d'expression

out1

Column

identifiant

civilite

categorie_professionnelle

nom

prenom

commune

profession

specialite

type_identifiant

mode_exercice

identifiant_organisation

identifiant_1

row3

Clé d'expr.

out1.identifiant

Column

Identifiant

Code_specialite

Var

out2

Expression

out1.identifiant

out1.civilite

out1.categorie_professionnelle

out1.nom

out1.prenom

out1.mode_exercice

row3.Code_specialite

out1.identifiant_organisation

row3.identifiant

out1.identifiant_1

Column

identifiant

civilite

categorie_professionnelle

nom

prenom

mode_exercice

Code_specialite

identifiant_organisation

id_pg

id_active

Editeur de Schéma

Editeur d'expression

out2

Column

identifiant

civilite

categorie_professionnelle

nom

prenom

mode_exercice

Code_specialite

identifiant_organisation

id_pg

id_activite

row4

Ci   d'expr.

out2.Code_specialite

Column

Code_specialite

Fonction

Specialite

Var

out3

Expression

out2.identifiant

out2.civilite

out2.categorie_professionnelle

out2.nom

out2.prenom

out2.mode_exercice

row4.Fonction

row4.Specialite

out2.identifiant_organisation

Column

identifiant

civilite

categorie_professionnelle

nom

prenom

mode_exercice

Fonction

Specialite

identifiant_organisation

Editeur de Sch  ma

Editeur d'expression

out2

Colonne	Cl��	Type	☒	N.	Mod��le de date (C...	Length	Precision	D��faut	Comment...
identifiant	☒	String	☒				0		
civilite	☐	String	☒			8	0		
categorie_professionnelle	☐	String	☒			5	0		
nom	☐	String	☒			13	0		
prenom	☐	String	☒			13	0		
mode_exercice	☐	String	☒			8	0		

out3

Colonne	Cl��	Type	☒	N.	Mod��le de date (C...	Length	Precision	D��faut	Comment...
identifiant	☒	String	☒				0		
civilite	☐	String	☒			8	0		
categorie_professionnelle	☐	String	☒			5	0		
nom	☐	String	☒			13	0		
prenom	☐	String	☒			13	0		
mode_exercice	☐	String	☒			8	0		

cloudera_HDFS(tHDFSOutput_1)

Param  tres simples

Param  tres avanc  s

Param  tres dynamiques

View

Documentation

Type de propri  t  

R  f  rentiel

HDFS:cloudera_HDFS

Sch  ma

Built-In

Modifier le sch  ma

Sync colonnes

☐ Utiliser une connexion existante

Version

Distribution

Cloudera

Version

Cloudera CDH5.12(YARN mode)

Sch  ma

Sch  ma

HDFS

Connexion

URI du NameNode

"hdfs://quickstart.cloudera:8020"

☒ Utiliser le nom d'h  te du n  ud de donn  es

Authentification

☐ Utiliser l'authentification de Kerberos

Utilisateur

"cloudera"

Nom de fichier

"/user/hive/warehouse/dim_professionnel/dim_professionnel.csv"

Type de fichier

Type

Fichier texte

Action

Ecraser

S  parateur de lignes

"\n"

S  parateur de champs

","

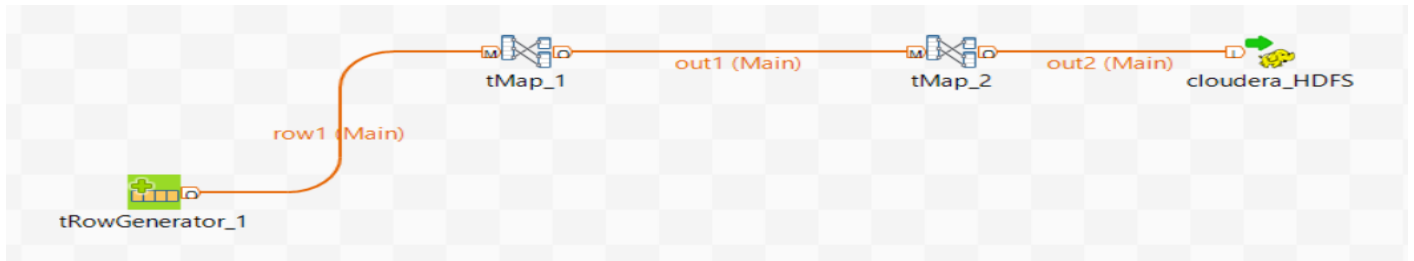
☐ Encodage personnalis  

Compression

☐ Compresser les donn  es

☒ Inclure l'en-t  te

2.1.6. Dim_temps



Colonne dateKey	Type Integer	Fonctions Numeric.sequenc...	Variables d'envir... sequence identifi...	Aperçu	
--------------------	-----------------	---------------------------------	--	--------	--

Colonne Nombre de lignes pour RowGenerator 18250

row1

Column

dateKey

Var

out1

Expression

row1.dateKey

TalendDate.addDate(... TalendDate.parseDate("...

Column

dateKey

fullDate

Éditeur de Schéma

Éditeur d'expression

row1

Colonne	Clé	Type	☒ N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
dateKey	☐	Integer	☒					

out1

Colonne	Clé	Type	☒ N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
dateKey	☐	Integer	☒					
fullDate	☐	Date	☒	"yyyy-MM-dd"				

out1

Column

dateKey

fullDate

Var

out2

Expression

out1.dateKey

out1.fullDate != null ? ... out1.fullDate ... : null

TalendDate.getPartOfDate("DAY_OF_MONTH", out1.fullDate)

TalendDate.getPartOfDate("MONTH", out1.fullDate)

TalendDate.getPartOfDate("YEAR", out1.fullDate)

TalendDate.formatDateLocale(... "EEEE", ... out1.fullDate)

TalendDate.formatDateLocale(... "MMMM", ... out1.fullDate)

((TalendDate.getPartOfDate("MONTH", out1.fullDate)) / 6) + 1

Column

id_date

fullDate

day

month

year

nom_jour

nom_mois

trimestre

Editeur de Schéma

Editeur d'expression

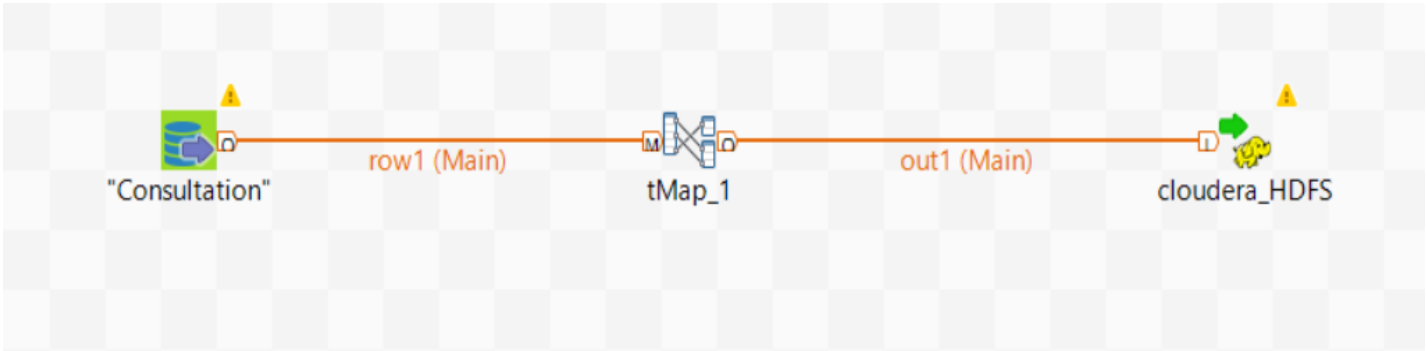
out1

Colonne	Clé	Type	N.	Modèle de date (C...)	Length	Precision	Défaut	Comment...
dateKey		☐ Integer	☒					
fullDate		☐ Date	☒	"yyyy-MM-dd"				

out2

Colonne	Clé	Type	N.	Modèle de date (C...)	Length	Precision	Défaut	Comment...
id_date		☒ Integer	☒					
fullDate		☐ Date	☒	"yyyy-MM-dd"				
day		☐ Integer	☒					
month		☐ Integer	☒					
year		☐ Integer	☒					
nom_jour		☐ String	☒					

2.1.7. Dim_Consultation



row1

Column

Num_consultation

Id_mut

Id_patient

Id_prof_sante

Code_diag

Motif

Date

Heure_debut

Heure_fin

Find :

Var

Mapping auto

out1

Expression

row1.Num_consultation

row1.Id_patient

row1.Id_prof_sante

row1.Code_diag

row1.Motif

row1.Date

row1.Heure_debut

row1.Heure_fin

(int)(... TalendDate.parseDate("HH:mm:ss", ro...

Column

Num_consultation

Id_patient

Id_prof_sante

Code_diag

Motif

Date

Heure_debut

Heure_fin

duree_h

Editeur de Schéma

Editeur d'expression

row1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
Num_consultation	☒	int	☐		10	0		
Id_mut	☐	Integer	☐		10	0		
Id_patient	☐	Integer	☒		10	0		
Id_prof_sante	☐	String	☒		21474836...	0		
Code_diag	☐	String	☒		21474836...	0		
Motif	☐	String	☒		21474836...	0		

out1

Colonne	Clé	Type	N.	Modèle de date (C...	Length	Precision	Défaut	Comment...
Num_consultation	☒	int	☐		10	0		
Id_patient	☐	Integer	☒		10	0		
Id_prof_sante	☐	String	☒		21474836...	0		
Code_diag	☐	String	☒		21474836...	0		
Motif	☐	String	☒		21474836...	0		
Date	☐	Date	☒	"yyyy-MM-dd"	13	0		

cloudera_HDFS(tHDFSOutput_1)

Paramètres simples

Paramètres avancés

Paramètres dynamiques

View

Documentation

Type de propriété

Référentiel

HDFS:cloudera_HDFS

Schéma

Built-In

Modifier le schéma

Sync colonnes

Utiliser une connexion existante

Version

Distribution

Cloudera

Version

Cloudera CDH5.12(YARN mode)

Schème

HDFS

Connexion

URI du NameNode

hdfs://quickstart.cloudera:8020

Utiliser le nom d'hôte du nœud de données

Authentification

Utiliser l'authentification de Kerberos

Utilisateur

cloudera

Nom de fichier

/user/hive/warehouse/dim_consult/dim_consultation.csv

Type de fichier

Type

Fichier texte

Action

Ecraser

Séparateur de lignes

\n

Séparateur de champs

,

Encodage personnalisé

Compression

Compresser les données

Inclure l'en-tête

2.2. Architectural Justification of the Ingestion Strategy

Why was it decided to use HDFS as well as native tHDFSOutput components instead of traditional database storage?

This platform employs the Hadoop Distributed File System (HDFS) as the basic landing zone in order to maximize storage scalability and file tracking fault tolerance in the platform. Through using the native tHDFSOutput elements (cloudera HDFS) in Talend, the ingestion process writes the data blocks into distributed clusters without any network or memory bottlenecks associated with the row-by-row database writing process.

Why was it chosen to ingest the data into separated raw text files instead of binary formats?

Synchronizing various relational and flat database tables in comma-separated text format ensures maximum platform independence and excellent traceability. Storing the data in uncompressed text format allows simultaneous access by other processes, including secondary processing scripts, internal test cases, and automated metadata evaluation utilities. Hive tables can also be created directly from these files.

The following image provides an overview of our folder containing the files sent from Talend, captured directly from the Cloudera HDFS Directory Browser :

Browse Directory

/user/hive/warehouse

Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxrwxrwx	cloudera	supergroup	0 B	Mon Jun 08 10:25:26 +0200 2026	0	0 B	dim_consult
drwxrwxrwx	cloudera	supergroup	0 B	Sat Jun 06 15:27:02 +0200 2026	0	0 B	dim_decès
drwxrwxrwx	cloudera	supergroup	0 B	Wed Jun 03 17:28:26 +0200 2026	0	0 B	dim_diagnostic
drwxrwxrwx	cloudera	supergroup	0 B	Wed Jun 03 17:28:48 +0200 2026	0	0 B	dim_etablissement
drwxrwxrwx	cloudera	supergroup	0 B	Mon Jun 08 15:35:50 +0200 2026	0	0 B	dim_hospital
drwxrwxrwx	cloudera	supergroup	0 B	Mon Jun 08 15:44:36 +0200 2026	0	0 B	dim_hospitalisation
drwxrwxrwx	cloudera	supergroup	0 B	Wed Jun 03 17:27:29 +0200 2026	0	0 B	dim_localisation
drwxrwxrwx	cloudera	supergroup	0 B	Mon Jun 08 15:56:08 +0200 2026	0	0 B	dim_patient
drwxrwxrwx	cloudera	supergroup	0 B	Tue Jun 09 16:25:27 +0200 2026	0	0 B	dim_professionnel
drwxrwxrwx	cloudera	supergroup	0 B	Mon Jun 08 09:41:16 +0200 2026	0	0 B	dim_satisfaction
drwxrwxrwx	cloudera	supergroup	0 B	Mon Jun 08 09:47:31 +0200 2026	0	0 B	dim_temps
drwxrwxrwx	cloudera	supergroup	0 B	Tue Jun 09 16:28:39 +0200 2026	0	0 B	projet.db

3. Measurement Table Design & Calculation Logic

3.1. Table Materialization for Measurements

Instead of reducing completely independent healthcare activities into a single and overly complex model, the warehouse uses the multi-fact Galaxy Schema approach ([Appendix B](#)). Four highly focused fact tables that represent the key measurements are divided and stored separately for maximum storage and vectorization capabilities:

- **fact_hospitalisation:** Specifically designed to track prolonged inpatient stays.
- **fact_consultation:** Created to log immediate transactions in terms of ambulatory outpatient consultation procedures.
- **fact_deces:** Designed to register national mortality records and demographic trends.
- **fact_satisfaction:** Constructed as an extensive matrix to track quantitative patient services and satisfaction ratings.

To optimize analytical query processing and eliminate cluster overhead, the measurement layer leverages Hive dynamic partitioning. Configured under nonstrict and sort dynamic partition optimization settings, the target tables are automatically isolated into efficient physical directories on HDFS: fact_hospitalisation and fact_consultation are partitioned by year and month, while fact_deces and fact_satisfaction are partitioned strictly by year. [1]

Architectural Justification for Fact-Table Partitioning: As part of a deliberate strategy, we decided to use Dynamic Partitioning and Sort-Merge Bucketing right off these fact tables with high cardinality rather than those dimensions with low cardinality, such as patients' demographics and location details. In a Hadoop-based environment, when we partition smaller dimension tables, it leads to what is called the "Small Files Problem" in HDFS; this will result in heavy load on the NameNode with regard to its metadata handling capability as well as reduced performance. This approach helps us to do Partition Pruning using the Hive query optimizer.

4. Data Materialization and Operational Query Compilation in Apache Hive (Hue).

Having successfully ingested the raw structured blocks using the Talend Data Ingestion Layer, the data blocks now sit safely in the HDFS infrastructure layer. At the next level of our infrastructure development cycle, the focus moves into the Cloudera ecosystem using the Hue web portal. This is our control console through which we compile and run all our HiveQL scripts that will develop our analytical warehouse layer.

Introducing Apache Hive to this process enables us to convert our flat data storage files to a very relational Galaxy schema. Through these queries performed in Hue, Cloudera's distributed computing capabilities come into play to perform large-scale analytical processing, complex multi-dimensional table lookups, schema structure enforcement, and localized dynamic partitioning within storage.

Our HiveQL scripts screenshots have been organized in order of operation as follows: database creation, partitioning optimization, and finally the validation of data integrity.

The following is the structured listing of the SQL scripts in Hue together with their rationale:

4.1. HiveQL Scripts.

4.1.1. CreateHospitalisation & InsertHospitalisation

```
CREATE TABLE IF NOT EXISTS fact_hospitalisation (  
    id_hospitalisation    BIGINT,  
    id_patient            BIGINT,  
    id_diag               BIGINT,  
    id_etablissement      BIGINT,  
    nb_hospitalisations   INT,  
    duree_sejour_jours    BIGINT  
)  
PARTITIONED BY (annee BIGINT, mois BIGINT)  
STORED AS ORC;
```

```

SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;
SET hive.exec.max.dynamic.partitions = 10000;
SET hive.exec.max.dynamic.partitions.pernode = 5000;
SET hive.optimize.sort.dynamic.partition = true;

INSERT INTO TABLE fact_hospitalisation
PARTITION (annee, mois)

SELECT
    h.num_hospitalisation           AS id_hospitalisation,
    h.id_patient                    AS id_patient,
    COALESCE(d.id_diag, -1)         AS id_diag,
    COALESCE(e.id_etablissement, -1) AS id_etablissement,
    1                               AS nb_hospitalisations,
    h.jour_hospitalisation          AS duree_sejour_jours,

    CAST(t.year AS BIGINT)          AS annee,
    CAST(t.month AS BIGINT)         AS mois

FROM dim_hospital h
INNER JOIN dim_temps t
    ON h.date_entree = t.fulldate
LEFT JOIN dim_diagnostic d
    ON h.code_diagnostic = d.code_diag
LEFT JOIN dim_etablissement e
    ON h.identifiant_organisation = e.identifiant_organisation

DISTRIBUTE BY (annee, mois);

```

Result of the query's:

Bases de données > projet > fact_hospitalisation_v2

No description available

Aperçu Colannes (8) Exemple Détails

	fact_hospitalisation_v2.id_hospitalization	fact_hospitalisation_v2.num_hospitalisation	fact_hospitalisation_v2.id_patient	fact_hospitalisation_v2.identifiant_organise
1	1	10546	29620	F010000107
2	2	52968	21126	F010000180
3	3	46019	4572	F010000198
4	4	59194	60237	F010002236
5	5	75758	53831	F010006500
6	6	70998	42859	F010007870
7	7	10728	34974	F010008050
8	8	63219	41726	F010009041
9	9	59938	83931	F010010593
10	10	71143	36981	F010010965
11	11	41730	17807	F010011468
12	12	12172	11346	F010780310

4.1.2. CreateConsultation & InsertConsultation

```
CREATE TABLE IF NOT EXISTS fact_consultation (  
    id_consult          BIGINT,  
    id_patient          BIGINT,  
    id_diag             BIGINT,  
    id_etablissement    BIGINT,  
    nb_consultations    INT,  
    duree_min           INT,  
    motif               STRING  
)  
PARTITIONED BY (  
    annee               BIGINT,  
    mois                BIGINT  
)  
STORED AS ORC;
```

```

INSERT INTO TABLE fact_consultation
PARTITION (annee, mois)

SELECT
    c.num_consultation           AS id_consult,
    c.id_patient                AS id_patient,
    COALESCE(d.id_diag, -1)     AS id_diag,
    COALESCE(e.id_etablissement, -1) AS id_etablissement,
    1                           AS nb_consultations,
    CAST(c.duree_h * 60 AS INT)  AS duree_min,
    c.motif                     AS motif,

    CAST(t.year AS BIGINT)      AS annee,
    CAST(t.month AS BIGINT)     AS mois
FROM dim_consultation c
INNER JOIN dim_temps t

```

```

    ON c.date = t.fulldate
LEFT JOIN dim_diagnostic d
    ON c.code_diag = d.code_diag
LEFT JOIN dim_professionnel p
    ON c.id_prof_sante = p.identifiant
LEFT JOIN dim_etablissement e
    ON p.identifiant_organisation = e.identifiant_organisation

DISTRIBUTE BY annee, mois;|

```

Result of the query's:

Bases de données > projet > fact_consultation_v2					▶ ↺ ⬆ ×
No description available					
Aperçu	Colonnes (8)	Exemple	Détails		
	fact_consultation_v2.num_consultation	fact_consultation_v2.id_patient	fact_consultation_v2.id_prof_sante	fact_consultation_v2.id_diag	fact_consultation_v2.id
17	1059115338	37233	10000001676	7775	R10000000617634
18	1059080000	9708	10000001676	1436	R10000000617634
19	1059719976	14994	10000001676	13013	R10000000617634
20	1059593129	95852	10000001676	15277	R10000000617634
21	1059471974	63477	10000001676	776	R10000000617634
1	1059065457	87511	10000001056	12635	F970300026
2	1059196805	62858	10000001056	7293	F970300026
3	1059126952	61440	10000001056	681	F970300026
4	1059886703	29501	10000001056	1273	F970300026
5	1059970158	35796	10000001056	1835	F970300026
6	1059678970	62844	10000001056	64	F970300026
48	1059721693	93922	10000003334	540	F970107249

4.1.3. CreateDeaths & InsertDeaths

```
CREATE TABLE fact_decès (id_date BIGINT, id_localisation BIGINT, nb_decès BIGINT)
PARTITIONED BY (year INT) STORED AS ORC;

SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;

INSERT INTO TABLE fact_decès
PARTITION(year)
SELECT
    t.id_date,
    l.id_localisation,
    COUNT(*) AS nb_decès,
    t.year
FROM dim_decès d
JOIN dim_temps t
    ON d.date_decès = t.fulldate
JOIN dim_localisation l
    ON d.code_lieu_decès = l.code_postal

GROUP BY
    t.id_date,
    l.id_localisation,
    t.year;
```

Result of the query's:

Bases de données > projet > fact_decès_v2

No description available

Aperçu Colonne (6) Exemple Détails

	fact_decès_v2.id_decès	fact_decès_v2.numero_acte_decès	fact_decès_v2.id_localisation	fact_decès_v2.id_date_decès	fact_decès_v2.nb_decès	fact_decès_v2.year
1	18989384	14	12822	6587	1	1004
2	18989385	13	12822	6590	1	1004
3	18989386	16	12822	6592	1	1004
4	18989387	19	12822	6597	1	1004
5	18989388	21	12822	6594	1	1004
6	18989390	17	12822	6597	1	1004
7	18989391	27	12822	6602	1	1004
8	18989392	22	12822	6597	1	1004
9	18989393	24	12822	6600	1	1004
10	18989394	4	12822	6576	1	1004
11	18989396	2	12822	6576	1	1004
12	18989398	9	12822	6583	1	1004

4.1.2. CreateSatisfaction & InsertSatisfaction

```
CREATE TABLE fact_satisfaction (  
    id_date BIGINT,  
    id_localisation BIGINT,  
    fitness BIGINT,  
    fitness_geo BIGINT,  
  
    score_all_rea DOUBLE,  
    score_accueil DOUBLE,  
    score_pecinf DOUBLE,  
    score_pecmed DOUBLE,  
    score_chambre DOUBLE,  
    score_repas DOUBLE,  
    score_sortie DOUBLE,  
  
    nb_rep BIGINT,  
    id_satisfaction BIGINT
```

```
)  
PARTITIONED BY (year BIGINT)  
STORED AS ORC;
```

```

SET hive.exec.dynamic.partition=true;
SET hive.exec.dynamic.partition.mode=nonstrict;

INSERT OVERWRITE TABLE fact_satisfaction PARTITION(year)

SELECT
    t.id_date,
    l.id_localisation,

    s.finess,
    s.finess_geo,

    s.score_all_rea_ajust,
    s.score_accueil_rea_ajust,
    s.score_pecinf_rea_ajust,
    s.score_pecmed_rea_ajust,

    s.score_chambre_rea_ajust,
    s.score_repas_rea_ajust,
    s.score_sortie_rea_ajust,

    s.nb_rep_score_all_rea_ajust,
    s.id_satisfaction,

    t.year

FROM dim_satisfaction s

JOIN dim_temps t
ON s.date_join = t.fulldate

JOIN dim_localisation l
ON TRIM(UPPER(s.region)) = TRIM(UPPER(l.reg_nom));

```

Result of the query's:

Bases de données > projet > fact_satisfaction_v2

No description available

Aperçu

Colonnes (15)

Exemple

Détails

	fact_satisfaction_v2.id_satisfaction	fact_satisfaction_v2.finess	fact_satisfaction_v2.finess_geo	fact_satisfaction_v2.id_localisation	fact_satisfaction_v2.id_date
1	1	10780054	10000024	416650	7305
2	2	10780062	10000032	416650	7305
3	3	10780096	10000065	416650	7305
4	4	10008407	10005239	416650	7305
5	5	10780195	10780195	416650	7305
6	6	10780203	10780203	416650	7305
16	16	30780092	30000061	416650	7305
17	17	30780100	30000079	416650	7305
18	18	30780118	30000087	416650	7305
19	19	30780548	30780548	416650	7305
20	20	30781116	30781116	416650	7305
21	21	30785430	30785430	416650	7305

Following developing our fact tables, we implemented the structural dimensions, which are very important and need to be setup first since they hold the primary keys. In order to confirm this, we need to know that the dimensional structure, including time, localization, diagnosis, and facilities, is properly set up.

Bases de données > projet > dim_consultation

No description available

Aperçu

Colonnes (9)

Exemple

Détails

	dim_consultation.num_consultation	dim_consultation.id_patient	dim_consultation.id_prof_sante	dim_consultation.code_diag	dim_consultation.motif
1	1059023406	272	10004985825	P041	Soins dentaires
2	1059023407	1071	10005233548	N744	Consultation
3	1059023408	1285	10101362548	S92700	Consultation
4	1059023409	3723	10003697488	M2532	Consultation
5	1059023410	3841	10000436864	M512	Consultation
6	1059023411	4092	10002478203	J069	Consultation
7	1059023412	4164	10003737219	L752	Consultation
8	1059023413	4477	10003330551	M0227	Panoramique dentaire
9	1059023414	4709	10100154573	M4140	Consultation
10	1059023415	4789	10001057420	Y613	Osteodensitometrie
11	1059023416	5165	10002965514	U99053	Consultation
12	1059023417	5465	10000829753	N433	Exploration fonctionnelle respiratoire

Bases de données > projet > dim_deces



No description available

Aperçu Colonne (4) Exemple Détails

	dim_deces.id_deces	dim_deces.date_deces	dim_deces.code_lieu_deces	dim_deces.numero_acte_deces
1	18989384	2018-01-13	1004	14
2	18989385	2018-01-16	1004	13
3	18989386	2018-01-18	1004	16
4	18989387	2018-01-23	1004	19
5	18989388	2018-01-20	1004	21
6	18989390	2018-01-23	1004	17
7	18989391	2018-01-28	1004	27
8	18989392	2018-01-23	1004	22
9	18989393	2018-01-26	1004	24
10	18989394	2018-01-02	1004	4
11	18989396	2018-01-02	1004	2
12	18989398	2018-01-09	1004	9
13	18989399	2018-01-03	1004	8

Bases de données > projet > dim_diagnostic



No description available

Aperçu Colonne (3) Exemple Détails

	dim_diagnostic.id_diag	dim_diagnostic.code_diag	dim_diagnostic.diagnostic
1	1	A066	Abces ambien du cerveau
2	2	A064	Abces ambien du foie
3	3	A065	Abces ambien du poumon
4	4	K610	Abces anal
5	5	K612	Abces anorectal
6	6	L028	Abces cutane
7	7	L020	Abces cutane
8	8	L023	Abces cutane
9	9	L021	Abces cutane
10	10	L022	Abces cutane
11	11	L024	Abces cutane
12	12	L029	Abces cutane

Bases de données > projet > dim_patient



No description available

Aperçu Colonne (9) Exemple Détails

	dim_patient.id_patient	dim_patient.ville	dim_patient.code_postal	dim_patient.date	dim_patient.groupe_sanguin	dim_patient.poid	dim_patient.tai
1	1	THIONVILLE	57100	1980-06-04	O+	54.3	162
2	2	BERGERAC	24100	2015-01-07	O+	29.7	130
3	3	SALON-DE-PROVENCE	13300	2009-08-08	O+	52.8	136
4	4	MÉRIGNAC	33700	1958-06-10	B+	95.2	161
5	5	LES LILAS	93260	1965-04-10	O+	105	168
6	6	CARCASSONNE	11000	1939-08-12	O+	78.9	170
7	7	PIERREFITTE-SUR-SEINE	93380	1952-05-07	A+	101.3	170
8	8	VIENNE	38200	1947-02-08	O-	49.7	163
9	9	SAINTE-MARIE	97438	1926-05-01	B+	68	177
10	10	LE MANS	72100	1930-03-09	O+	91.5	182
11	11	GRASSE	6130	1986-08-03	B+	56.5	168
12	12	COGNAC	16100	1967-09-07	B+	94.7	168

Bases de données > projet > dim_hospital

No description available

Aperçu

Colonnes (8)

Exemple

Détails

	dim_hospital.num_hospitalisation	dim_hospital.id_patient	dim_hospital.identifiant_organisation	dim_hospital.code_diagnostic	dim_hospital.suite_diagnostic_coi
1	10546	29620	F010000107	S02800	Fracture de l'alveole dentaire; fermee
2	52968	21126	F010000180	Q902	Trisomie 21; translocation
3	46019	4572	F010000198	R192	Peristaltisme apparent
4	59194	60237	F010002236	M1125	Autre chondrocalcinose; region pelvi
5	75758	53831	F010006500	Y648	Administration par d'autres moyens c
6	70998	42859	F010007870	C754	Tumeur maligne du corpuscule carot
7	10728	34974	F010008050	S62471	Fractures multiples de sites multiples
8	63219	41726	F010009041	S92901	Fracture ouverte du pied; sans precis
9	59938	83931	F010010593	M4850	Tassement vertebra; non classe aille
10	71143	36981	F010010965	D684	Carence acquise du facteur de coagu
11	41730	17807	F010011468	D512	Carence en transcobalamine II
12	12172	11346	F010780310	Q716	Main en pince de homard

Bases de données > projet > dim_localisation

No description available

Aperçu

Colonnes (8)

Exemple

Détails

	dim_localisation.code_postal	dim_localisation.commune	dim_localisation.adresse	dim_localisation.pays	dim_localisation
1	1400		64 Rue COMMERSON 01400	France	1
2	1440	Viriat	900 Route DE PARIS 01440 Viriat	France	1
3	1300	Belley	Zone industrielle DU CORON 01300 Belley	France	1
4	1960	Péronnas	131 Rue JEAN MONNET 01960 Péronnas	France	1
5	1440		150 Route DE CRANGEAT 01440	France	1
6	1600	Reyrieux	Route VEISSIEUX LE HAUT 01600 Reyrieux	France	1
7	1110	Plateau d'Hauteville	Avenue FELIX MANGINI 01110 Plateau d'Hauteville	France	1
8	1420	Chanay	7 Rue DU CHÂTEAU 01420 Chanay	France	1
9	1300	Belley	52 Rue GEORGES GIRERD 01300 Belley	France	1
10	1012	Bourg-en-Bresse	Avenue DE MARBOZ 01012 Bourg-en-Bresse	France	1
11	1500		31 Avenue DU GENERAL DE GAULLE 01500	France	1
12	1100	Oyonnax	2 Route DE VEYZIAT 01100 Oyonnax	France	1

Bases de données > projet > dim_satisfaction

No description available

Aperçu

Colonnes (14)

Exemple

Détails

	dim_satisfaction.id_satisfaction	dim_satisfaction.finess	dim_satisfaction.finess_geo	dim_satisfaction.region	dim_satisfaction.date_join	dim_satisfaction.an
1	1	10780054	10000024	Auvergne-Rhône-Alpes	2020-01-01	2020
2	2	10780062	10000032	Auvergne-Rhône-Alpes	2020-01-01	2020
3	3	10780096	10000065	Auvergne-Rhône-Alpes	2020-01-01	2020
4	4	10008407	10005239	Auvergne-Rhône-Alpes	2020-01-01	2020
5	5	10780195	10780195	Auvergne-Rhône-Alpes	2020-01-01	2020
6	6	10780203	10780203	Auvergne-Rhône-Alpes	2020-01-01	2020
7	7	20000022	20000089	Hauts de France	2020-01-01	2020
8	8	20000063	20000162	Hauts de France	2020-01-01	2020
9	9	20000360	20000360	Hauts de France	2020-01-01	2020
10	10	20000253	20000394	Hauts de France	2020-01-01	2020
11	11	20000261	20000519	Hauts de France	2020-01-01	2020
12	12	20000287	20000535	Hauts de France	2020-01-01	2020

.....

5. Performance Assessment & Analytical Query Analysis

5.1. Target Analytical Queries Execution

The effectiveness of this Big Data solution will rely largely on its ability to address basic analytical healthcare needs. Utilizing the integrated «Projet» database, the process layer Hive analyzes complicated queries based on the dimensions of our model and four essential fact tables in order to satisfy crucial health monitoring needs:

- Patient consultation rate at facility X over period of time Y;
- Patient consultation rate in relation to diagnosis X over a period of time Y;
- Overall hospitalization rate of patients in a given period Y;
- Hospitalization rates of patients relative to diagnoses over a given period;
- Hospitalization rates by gender, by age;
- Consultation rate per professional;
- Number of deaths by location (region) and in 2019;
- Overall satisfaction rate by region for 2020;

5.1.1. Advanced HiveQL Optimization Techniques

In order to provide this information efficiently, avoiding computations on a row-by-row basis, the design of the query utilizes sophisticated HiveQL technique:

Advanced Aggregation Functions: Utilizes SUM, COUNT, and AVG functions to aggregate data directly based on the multi-fact structure [3].

5.2. Query Performance Testing Methodology & Benchmarking Environment

The correct assessment of the Cloudera CDH cluster processing efficiency was made by performing all scripts under the standard benchmarking procedure that used the Apache Hive engine:

Execution Engine Level: Hive use the MapReduce paradigm for optimizing the allocation of containers on cluster YARN memory partitions.

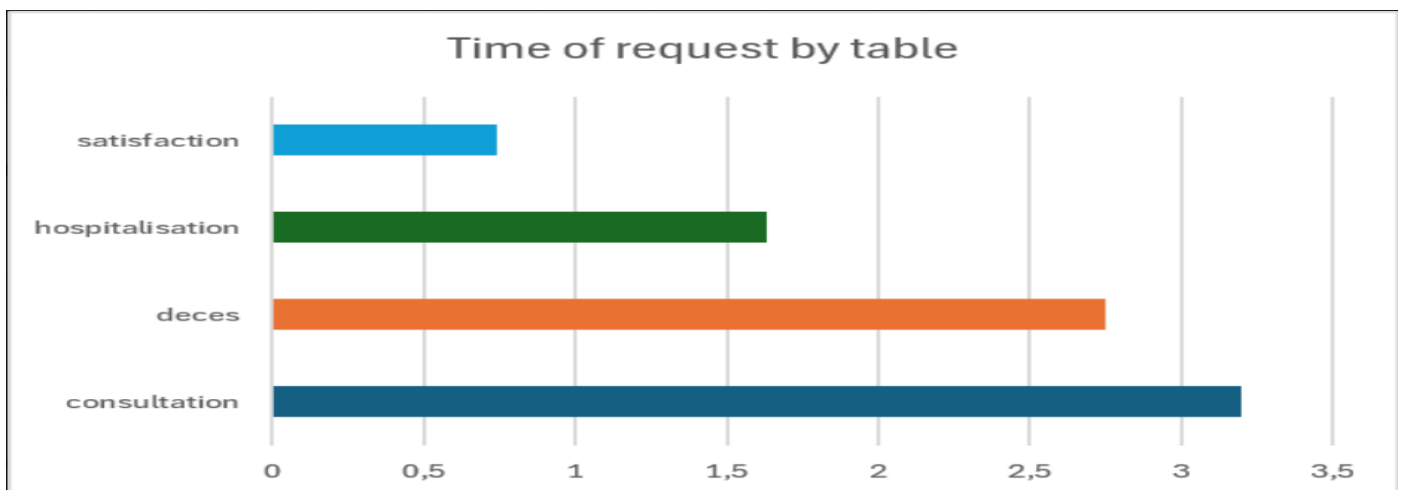
Multiple Iteration Log Results: In case of each selected query, 3 iterations were performed. The benchmark result obtained is the real average execution time of a query [2].

5.3. Comparative Runtime Performance & Response Time Analysis

Query used to fetch all those performances:

```
SELECT *  
FROM 'name_of_the_table'  
ORDER BY 'id_table' ASC  
LIMIT 100;
```

The following chart outlines the response times across our four core fact tables, highlighting how data volume and structural layout directly impact execution speed:



5.3.1. Technical Performance Drivers

Execution time speeds seen from the logs that scale as low as a few seconds are attributed to three major architectural attributes:

I/O Simplification through ORC Columnar Storage: The adoption of columnar ORC storage rather than row-based text storage reduces I/O operations because Hive can selectively load just the appropriate columns to calculate certain metrics (satisfaction score, number of consultations), thus reducing memory usage.

Temporal & Regional Partition Filters: Temporal and regional filters on partition enable Hive to filter HDFS directories where processing is required. This avoids unnecessary scanning of tables in irrelevant directories. [1]

5.3.2. Live Job Log Outputs

Below are the official task execution logs captured from the Hue interface during test query runtimes, confirming successful compilation and exact cumulative CPU tracking:

```
INFO : Starting task [Stage-1:MAPRED] in serial mode
INFO : Number of reduce tasks determined at compile time: 1
INFO : In order to change the average load for a reducer (in bytes):
INFO :   set hive.exec.reducers.bytes.per.reducer=<number>
INFO : In order to limit the maximum number of reducers:
INFO :   set hive.exec.reducers.max=<number>
INFO : In order to set a constant number of reducers:
INFO :   set mapreduce.job.reduces=<number>
INFO : number of splits:1
INFO : Submitting tokens for job: job_1781014946800_0001
INFO : The url to track the job: http://quickstart.cloudera:8088/proxy/application_1781014946800_0001/
INFO : Starting Job = job_1781014946800_0001, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1781014946800_0001/
INFO : Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1781014946800_0001
INFO : Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
INFO : 2026-06-09 07:41:49,722 Stage-1 map = 0%,   reduce = 0%
INFO : 2026-06-09 07:41:57,077 Stage-1 map = 100%,   reduce = 0%, Cumulative CPU 2.27 sec
INFO : 2026-06-09 07:42:02,359 Stage-1 map = 100%,   reduce = 100%, Cumulative CPU 3.2 sec
INFO : MapReduce Total cumulative CPU time: 3 seconds 200 msec
```

job_1781014946800_0001

```
INFO : In order to set a constant number of reducers:
INFO :   set mapreduce.job.reduces=<number>
INFO : number of splits:1
INFO : Submitting tokens for job: job_1781014946800_0002
INFO : The url to track the job: http://quickstart.cloudera:8088/proxy/application_1781014946800_0002/
INFO : Starting Job = job_1781014946800_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1781014946800_0002/
INFO : Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1781014946800_0002
INFO : Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
INFO : 2026-06-09 07:44:14,862 Stage-1 map = 0%,   reduce = 0%
INFO : 2026-06-09 07:44:20,072 Stage-1 map = 100%,   reduce = 0%, Cumulative CPU 1.88 sec
INFO : 2026-06-09 07:44:27,359 Stage-1 map = 100%,   reduce = 100%, Cumulative CPU 2.75 sec
INFO : MapReduce Total cumulative CPU time: 2 seconds 750 msec
INFO : Ended Job = job_1781014946800_0002
INFO : MapReduce Jobs Launched:
INFO : Stage-Stage-1: Map: 1   Reduce: 1   Cumulative CPU: 2.75 sec   HDFS Read: 3640029 HDFS Write: 2718 SUCCESS
INFO : Total MapReduce CPU Time Spent: 2 seconds 750 msec
```

job_1781014946800_0002

```
INFO : In order to limit the maximum number of reducers:
INFO :   set hive.exec.reducers.max=<number>
INFO : In order to set a constant number of reducers:
INFO :   set mapreduce.job.reduces=<number>
INFO : number of splits:1
INFO : Submitting tokens for job: job_1781014946800_0004
INFO : The url to track the job: http://quickstart.cloudera:8088/proxy/application_1781014946800_0004/
INFO : Starting Job = job_1781014946800_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1781014946800_0004/
INFO : Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1781014946800_0004
INFO : Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
INFO : 2026-06-09 07:46:35,292 Stage-1 map = 0%,   reduce = 0%
INFO : 2026-06-09 07:46:39,418 Stage-1 map = 100%,   reduce = 0%, Cumulative CPU 0.72 sec
INFO : 2026-06-09 07:46:44,610 Stage-1 map = 100%,   reduce = 100%, Cumulative CPU 1.63 sec
INFO : MapReduce Total cumulative CPU time: 1 seconds 630 msec
```

```

INFO : Total jobs = 1
INFO : Launching Job 1 out of 1
INFO : Starting task [Stage-1:MAPRED] in serial mode
INFO : Number of reduce tasks determined at compile time: 1
INFO : In order to change the average load for a reducer (in bytes):
INFO :   set hive.exec.reducers.bytes.per.reducer=<number>
INFO : In order to limit the maximum number of reducers:
INFO :   set hive.exec.reducers.max=<number>
INFO : In order to set a constant number of reducers:
INFO :   set mapreduce.job.reduces=<number>
INFO : number of splits:1
INFO : Submitting tokens for job: job_1781014946800_0006
INFO : The url to track the job: http://quickstart.cloudera:8088/proxy/application_1781014946800_0006/
INFO : Starting Job = job_1781014946800_0006, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1781014946800_0006/
INFO : Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1781014946800_0006
INFO : Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
INFO : 2026-06-09 07:49:16,643 Stage-1 map = 0%, reduce = 0%
INFO : 2026-06-09 07:49:21,848 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 0.74 sec

```

6. Conclusion & Deliverables Index

6.1 Conclusion & technical Challenges Overcome.

This deliverable being complete is an important step before switching to Power BI, since a solid big data architecture has been created inside Cloudera. Having converted ingestion streams into the structured galaxy schema in the Project database, we have achieved the creation of a scalable infrastructure required for the needs of the project.

Firstly, joining several tables at once led to skewed data distributions and even out-of-memory situations for master nodes; the problem was dealt with using optimization commands and dynamic partitions. Secondly, early data loss was fixed due to wrong type matches for strings on the staging level and dimensions; casting and normalization rules (like CAST and TRIM, UPPER) were added to HiveQL queries. [2]

6.2 Next Steps: Visualization and Data Storytelling (Deliverable 3)

Having optimized our schemas, columnar formats, and partitioning setups, our structure is fully prepared. We are now at a point where we are ready to move into **Deliverable 3: Dashboarding & Data Storytelling**

Our next step is to interface the optimized tables with our business intelligence visualizations. Making use of our pre-sorted named views and partition pruning capabilities.

Our dashboards will be designed to provide deliverable monitoring information for hospital executives based on the needs defined specifications.

6.3. Index of Delivered Scripts (Attached .zip Manifest File)

The different Talend jobs are encapsulated within a structured script archive:

[Jobs_Scripts.zip](#)

7. BIBLIOGRAPHY & REFERENCES

[1] **Cloudera Inc.** (May 2024). *Tips and Best Practices for Optimizing Hive Performance*. Cloudera Community Documentation.

Available at: <https://community.cloudera.com/t5/Community-Articles/Tips-and-best-practices-for-optimizing-Hive-performance/ta-p/385637> . Seen on June 02 2026.

[2] **Cloudera Inc.** (May 2022). *Optimizing Hive on Tez Performance*. Cloudera Technical Blog.

Available at: <https://www.cloudera.com/blog/technical/optimizing-hive-on-tez-performance.html> . Seen on June 04 2026.

[3] **Analytics Vidhya** (March 2022). *Performance Tuning Practices in Apache Hive*. Data Engineering Library.

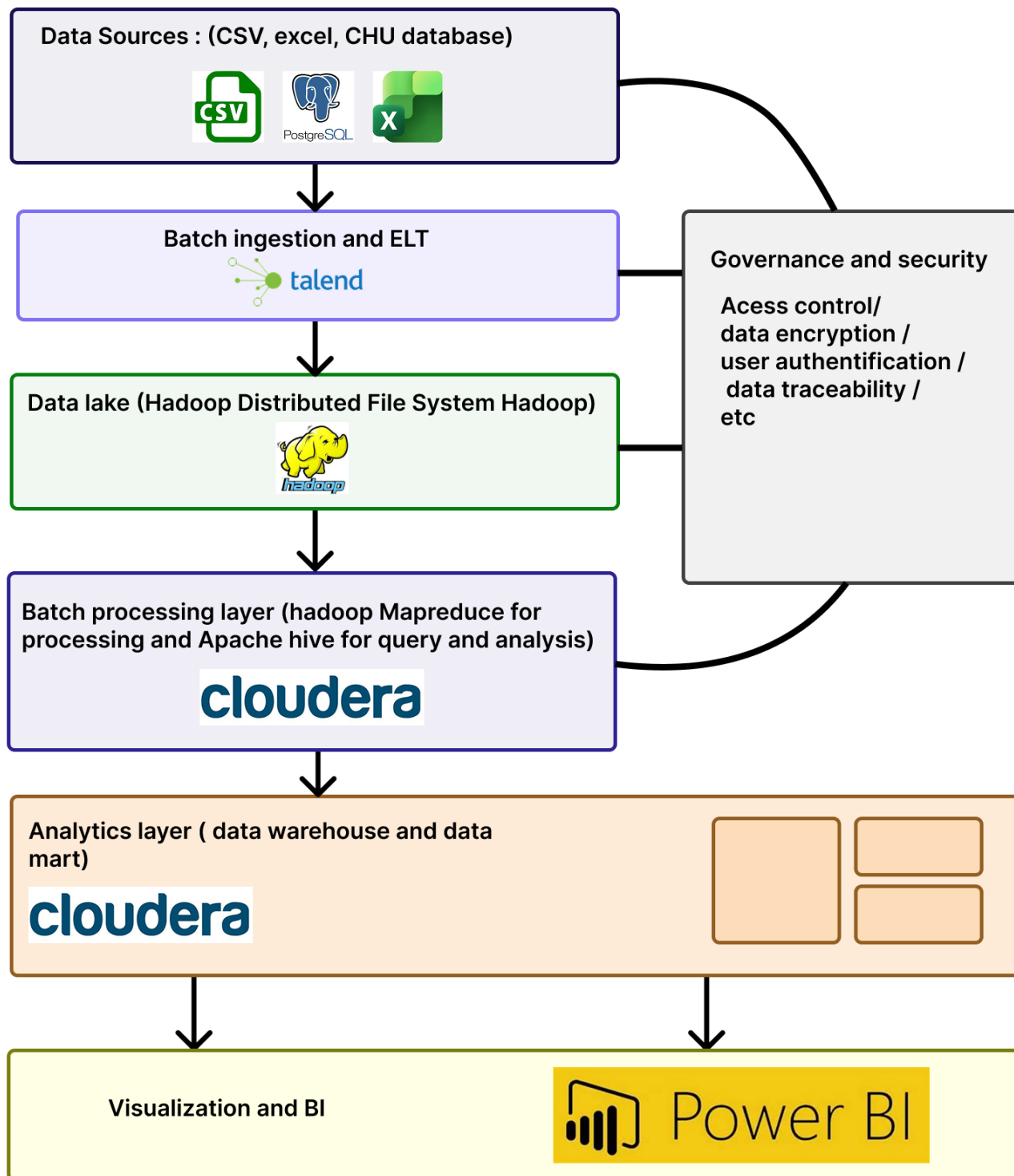
Available at: <https://www.analyticsvidhya.com/blog/2022/02/performance-tuning-practices-in-hive/> Seen on June 05 2026.

[4] **Talend Inc.** (March 2024). *Talend Open Studio for Big Data: Data Integration Architecture and Component Mapping Guide*. Talend Help Center Official Documentation.

Available at: <https://help.talend.com/r/en-US/cloud-data-integration/studio-user-guide/big-data-job-design-best-practices> . Seen on June 03 2026.

8. APPENDICES

8.1. Appendix A : Architecture.



8.2. Appendix B : Galaxy Schema.

